Computer Science & IT



Data Structure & programming

Linked List

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



- Topic — Add begin
- Topic — Addend
- Topic — Linked list problem
- Topic
- Topic

Slide

Topics to be Covered



- Topic
- Topic
- Topic
- Topic
- Topic

Add an element after given element

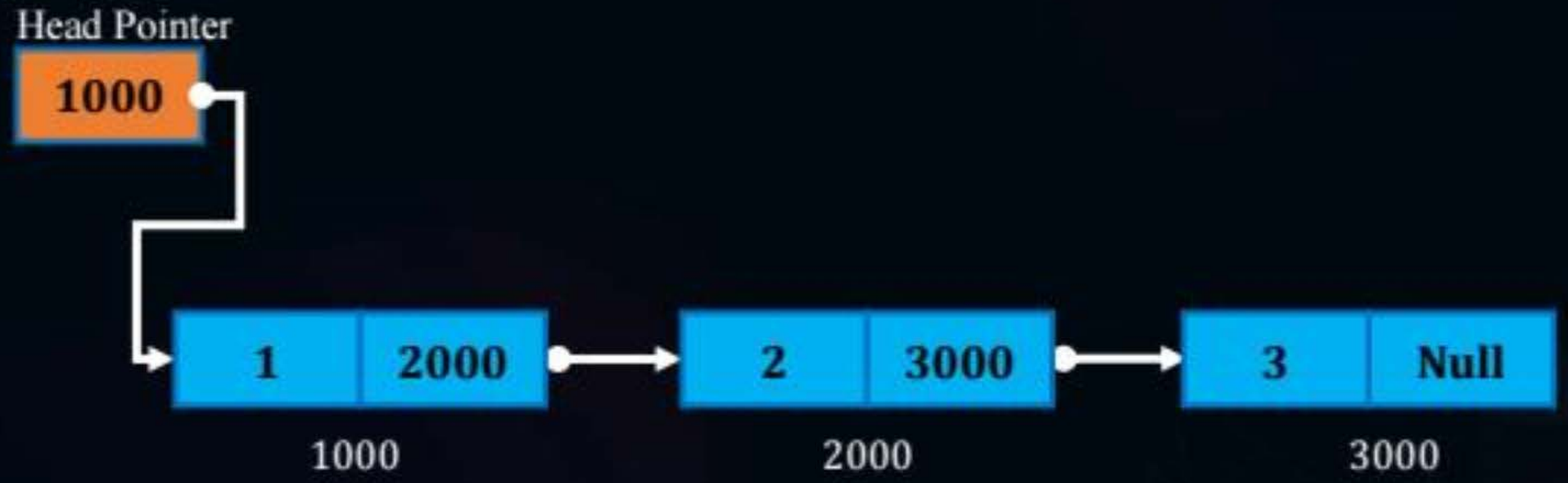
Delete element from list

problem

Slide



Topic : Single Linked List



Slide

Topic : Single Linked List



q

after 3 we want to insert 5

q→next



p 5000

Adding an element x after the element y

Nonempty list

if we first modify q→next then

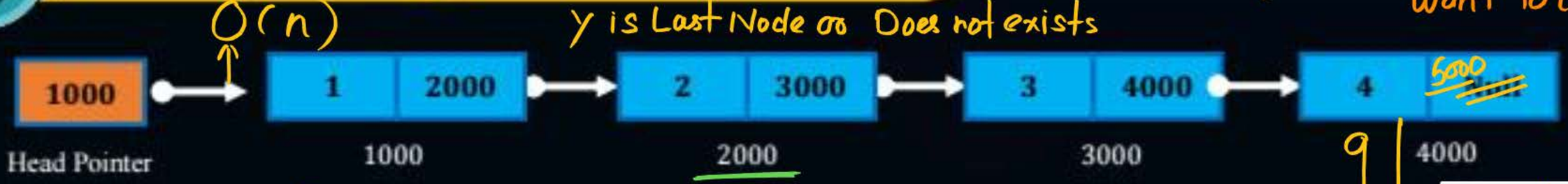
the address 4000 will be lost

① p→next = q→next

② q→next = p;

Topic : Single Linked List





after 3 we want to insert 5

P5000

```
void Addxafter(node *q, int x, int y) {
    node *p = getnode(x);
    if(q == NULL)
        return;
    while( q->next != NULL && q->data != y)
        q = q->next;
    if(q->data == y) {
        p->next = q->next;
        q->next = p;
    } else {
        printf("y does not exists");
        free(p);
    }
}
```

Slide

Topic : Single Linked List



if want to insert an element & pointer is given

for the Node **after** which insertion will happen.

Insert after 3 and you are given Address of 3

We dont required traversal. Independ of Length of list : O(1)

Slide

Topic : Single Linked List



deletey from the list

2 painter Solution

p → next = q → next

free(q)

if Last Node is deleted then Head painter will be modified.

Slide

Topic : Single Linked List




```
void ddelete( node * q, int y) {

    node * p = NULL;

    if( q == NULL)
        return;

    while( q->next != NULL && q->data != y){
        p = q;
        q = q->next;
    }

    if( q->data == y){
        p->next = q->next;
        free(q);
    }else
        printf("y is not present");

}
```

we can delete last node property

Slide

Topic : Single Linked List




```
void ddelete( node * q, int y) {

    node * p = NULL;

    if( q == NULL)
        return;

    while( q->next != NULL && q->data != y){
        p = q;
        q = q->next;
    }

    if( q->data == y){
        p->next = q->next;
        free(q);
    }else
        printf("y is not present");

}
```

Slide

Topic : Single Linked List



only one Node

P is NULL
NULL pointer dereference

```
void ddelete( node * q, int y) {

    node * p = NULL;

    if( q == NULL)
        return;

    while( q->next != NULL && q->data != y){
        p = q;
        q = q->next;
    }

    if (q->data == y) {
        p->next = q->next;
        free(q);
    }
    else
        printf("y is not present");

}
```

Slide

Topic : Single Linked List




```
void deleteby( node* q, int y) {
    node* p = NULL;
    if( q == NULL)
        return;
    else if( q->data == y)
    {
        p = q->next;
        HP = p;
        free(q);
        return;
    }
    else {
        while(q->next != NULL && q->data != y){
            p = q;
            q = q->next;
        }
        if( q->data == y) {
            p->next = q->next;
            free(q);
        }
        else
            printf("y Not present");
    }
}
```

List is NULL

if first Node deleted

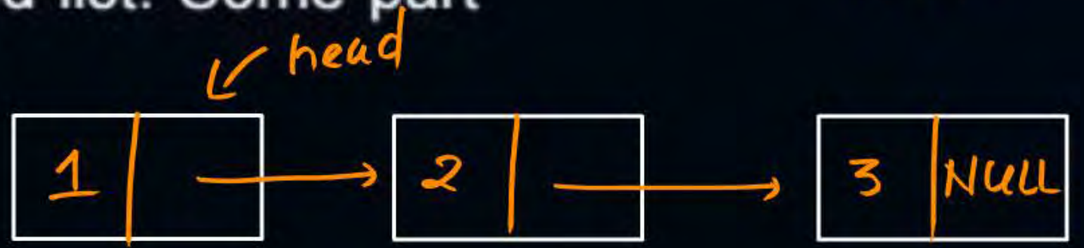
Slide

Single

The following C function takes a simply-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

2010

```
typedef struct node {
        int value;
        struct node *next;
}Node;
```

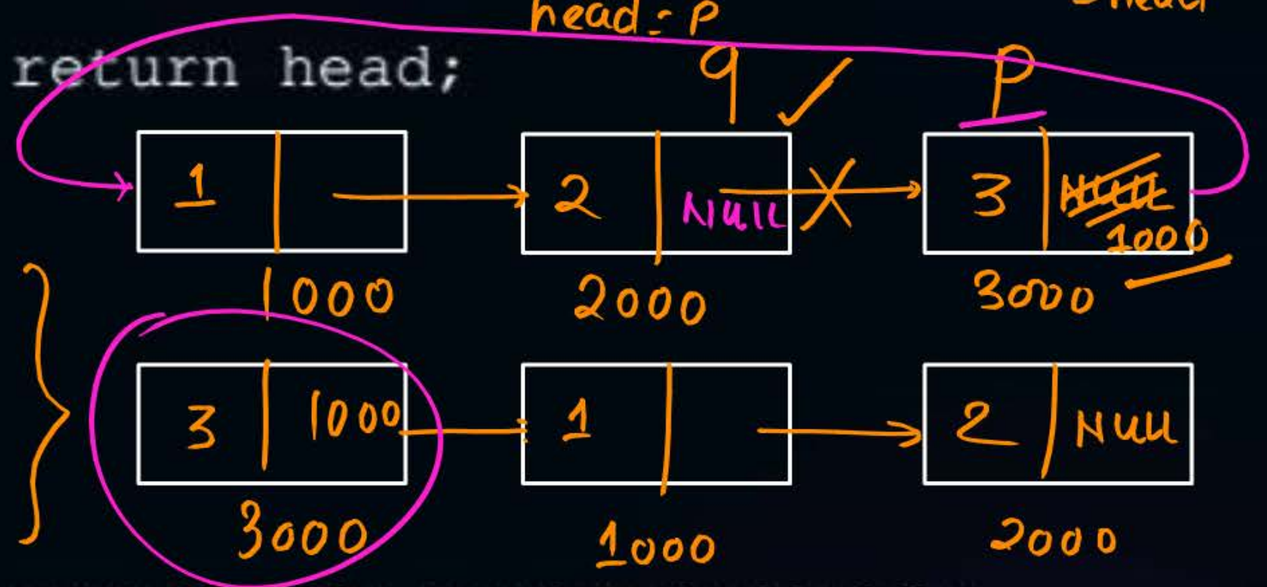




```
Node *move to front (Node *head) {
        Node *p, *q;
        if ((head == NULL) || (head - > next = = NULL))
                return head;
        q= NULL; p=head;
        while (p-> next! = NULL) {
                q = p;
                p = p -> next;
        }
        _____
        return head;
```

q→next = NULL ✓

P→next = head



Choose the correct alternative to replace the blank line.

(A) `q = NULL; p - > next = head; head = p;`

(B) `q -> next = NULL; head = p; p -> next = head;`

(C) `Head = p; p -> next = q;q -> next = NULL;` ✗

(D) `q -> next = NULL;` `p -> next = head;` `head = p;` ✓

Topic Question

The following C function takes a single-linked list of integers as a parameter and rearranges the element of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {
        int value;
        struct node *next;
};
```

Topic Question

```
void rearrange (struct node *list){
strucut node *p, *q;
int temp;
if (!list | | !(list-> next)) return;
p = list; q = list -> next;
while (q) {
temp = p ->value; p -> value = q -> value;
q -> value = temp; p = q ->next;
q = p ? p-> next: 0;

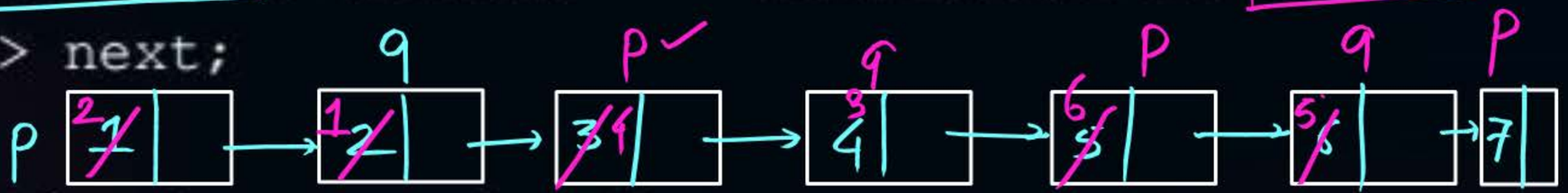
}
}
```

Empty

Single element

(A) 1, 2, 3, 4, 5, 6, 7
(B) 2, 1, 4, 3, 6, 5, 7
(C) 1, 3, 2, 5, 4, 7, 6
(D) 2, 3, 4, 5, 6, 7, 1

= gamb



q = p ? p->next : 0

true

if (p)
q = p->next;
else
q = 0;

Question

What is output of the program if the list contains 1→2 →3?

```
HP is global variable
void Dosomething(){
        node *q, *r, *s;
        q = HP;
        r = NULL;
                while( q!=NULL){
                        s = r;
                        r = q;
                        q = q→ link;
                        r→ link =s;
                }
                HP = r;
        }
```

HW question



Topic Question

Consider the function defined below.

```
struct item {
        int data;
        struct item * next;
};
int f(struct item *p) {
    return ((p == NULL) || (p->next == NULL) ||
        ((p->data <= p ->next -> data) &&
        f(p->next)));
}
```

For a given linked list p , the function f returns 1 if and only if

A the list is empty or has exactly one element

B the elements in the list are sorted in non-decreasing order of data value

C the elements in the list are sorted in non-increasing order of data value

D . not all elements in the list have the same data value

Slide

Weekend

Extend

1, 2, 3 4 ← Increasing

1, 1, 2, 4 ← Non decreasing

4, 3. 2. 1

4, 4, 2 2

a_i, a_{i+1}

$a_i <= a_{i+1}$

↑ Non-decreasing



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Insert x after y

Delete y from list

Slide

THANK - YOU

Slide